Reflection on Final Project: AI-Assisted Testing and Git Automation

Sanam Choudhary
Computer Science
DePaul University
Chicago, Illinois
schoud18@depaul.edu

***Abstract*—This report reflects on the experience of developing a final project using AI-assisted testing and Git automation. It discusses the methodology, results, and insights gained throughout the process, including technical challenges, debugging experiences, and the reliability of AI in generating unit tests versus specification-based testing. The report also highlights patterns in code coverage improvement and provides recommendations for future enhancements, emphasizing what was learned about the strengths and limitations of AI tools in software engineering workflows.**

***Index Terms*—AI-assisted development, Java testing, JUnit, code coverage, Git automation, software reflection**

I. Introduction

The final project focused on exploring AI-assisted development tools for Java-based software testing. The primary objective was to evaluate the effectiveness of AI in generating JUnit tests, analyzing code coverage, and automating Git workflow tasks. Traditional software testing is labor-intensive, requiring developers to manually create test cases based on specifications. This project provided an opportunity to investigate whether AI could streamline testing, improve coverage, and offer actionable insights.

In addition to evaluating AI test generation, this project emphasized practical learning, including debugging complex Java code, integrating Maven for build management, and utilizing Jacoco for coverage reporting. Understanding these processes provided insight into the challenges of balancing automated tools with hands-on development practices.

II. Methodology

The project methodology consisted of multiple phases:

A. AI-Assisted Analysis and Test Generation

AI tools were used to parse Java source code and extract class structures, method signatures, and dependencies. Based on this analysis, the AI generated JUnit test skeletons intended to cover the identified methods. While the AI could automatically produce test files, many tests required manual adjustment to ensure correctness, particularly for methods with complex preconditions or side effects.

B. Manual Specification-Based Testing

To supplement AI-generated tests, specification-based testing was implemented manually. This approach involved reading the method specifications and creating targeted test cases that ensured accurate behavior. This manual process revealed gaps in AI-generated tests, demonstrating that while AI could accelerate test generation, human insight remains essential for reliability.

C. Coverage Analysis

Jacoco was used to generate coverage reports after each round of testing. These reports provided detailed metrics, including line, branch, and method coverage. AI-assisted tools helped visualize coverage gaps and suggested areas needing further testing, which guided the manual addition of tests.

D. Git Automation

AI tools were also employed to automate Git operations, including commits, branch management, and status checks. This workflow automation reduced manual effort and highlighted an effective integration of AI in project version control and team collaboration processes.

III. Results & Discussion

A. Analysis of Coverage Improvement Patterns

Code coverage improved significantly when AI analysis was combined with manual test creation. Initially, AI-generated tests provided limited coverage due to incomplete handling of edge cases. By iteratively adding specification-based tests guided by Jacoco reports, overall coverage increased. Patterns observed include:

- AI coverage suggestions often prioritized frequently called methods.
- Branch coverage remained lower than line coverage without manual intervention.
- Complex or state-dependent methods consistently required manual testing.

B. Insights from AI-Assisted Development

AI tools provided valuable benefits in specific areas:

- Automated parsing of Java code and extraction of method signatures was fast and reliable.

- Jacoco reports generated by AI allowed quick identification of untested areas.
- Git workflow automation reduced the risk of human error in commit and branch management.

Despite these advantages, AI-generated JUnit tests were often inconsistent or unclear. Many tests contained incorrect assertions or incomplete logic, making them unreliable without manual correction. As a result, specification-based testing remained the most dependable approach for ensuring correctness.

C. Technical Challenges and Learning Experiences

Several challenges emerged during the project:

- Integrating AI tools with existing Java and Maven projects required careful configuration. This was the hardest part of the entire project for me. I struggled a lot with trying to make the AI tools work in conjunction with Java projects.
- Debugging AI-generated tests was more time-consuming than writing manual tests due to ambiguous assertions and unexpected behavior.
- Understanding Jacoco coverage metrics and interpreting gaps in the context of project logic required careful analysis.

These challenges offered valuable learning opportunities, particularly in:

- Developing a structured testing workflow that combines AI analysis with manual verification.
- Gaining deeper insight into how automated tools interpret code and generate tests.
- Appreciating the importance of human oversight when relying on AI tools in critical development processes.

D. Recommendations for Future Enhancements

- Improve AI algorithms to better handle method preconditions, exceptions, and state-dependent logic.
- Introduce AI-assisted test suggestions rather than fully automated test generation, allowing developers to make final decisions.
- Continue leveraging AI for coverage analysis and Git automation while retaining manual specification-based testing for accuracy.
- Consider integrating additional AI-based static analysis tools to complement coverage data and identify potential logic flaws.

IV. Conclusion

This project provided hands-on experience with AI-assisted software testing and workflow automation. AI tools excelled in code analysis, coverage reporting, and Git management, but fell short in generating reliable unit tests. Specification-based testing remains essential for accurate, predictable validation of software behavior. Overall, the project highlighted both the strengths and limitations of AI in software development, offering practical lessons in integrating automation with human expertise for effective software engineering.

Acknowledgment

I would like to thank my course instructors and peers for their guidance and feedback throughout this project. Their insights were invaluable in navigating the challenges of AI-assisted development and testing. Also, shout-out Dr. Kim! I enjoyed this quarter a lot and felt like I learned valuable skills as a software engineer :) !!